

A Neuro-Symbolic Pipeline for Automated Synthesis and Verification of OCL Constraints

YUELOU DING^{✉*}, Department of Computer Science, Maynooth University, Ireland

HAO WU[✉], Department of Computer Science, Maynooth University, Ireland

Authoring OCL constraints manually in Model-Driven Engineering is error-prone. While Large Language Models offer a potential solution for automated synthesis, they suffer from structural and semantic hallucinations. Current LLM-based approaches primarily target statistical translation accuracy, lacking formal guarantees of semantic correctness. We propose a neuro-symbolic synthesis pipeline that couples LLM generation with a three-layer checking mechanism and a self-correction loop. Layer 1 enforces structural compliance via JSON Schema validation, and Layer 2 performs static type checking within the UML context. Crucially, Layer 3 translates the candidate OCL constraint into SMT formulas for strict semantic equivalence checking with the reference specification. To ensure decidability, we define a decidable OCL subset and introduce a novel bounded SMT encoding strategy. This strategy employs a value-condition separation mechanism to handle OCL's four-valued logic and abstracts collection algebra into counting functions with bounded quantifier elimination, reducing complex constraints to quantifier-free formulas. Evaluation on a benchmark of 137 OCL constraints demonstrates that the pipeline achieves a strict semantic equivalence rate of 98.54%, while maintaining computational tractability within standard bounded scopes.

CCS Concepts: • **Software and its engineering** → **Software verification**; • **Computing methodologies** → *Machine translation*.

Additional Key Words and Phrases: OCL synthesis, Neuro-Symbolic Methods, Formal Verification, Semantic Equivalence Checking, SMT Encoding

ACM Reference Format:

Yuelou Ding and Hao Wu. 2026. A Neuro-Symbolic Pipeline for Automated Synthesis and Verification of OCL Constraints. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 18 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

In Model-Driven Engineering (MDE), abstract models replace source code as the primary development artifacts [5]. To specify precise constraints within Unified Modeling Language (UML), the Object Constraint Language (OCL) [21] provides a declarative, formal syntax. OCL invariants are crucial for maintaining model consistency and verifying system properties beyond the expressive power of graphical diagrams [11]. However, authoring OCL manually is an error-prone task. The language's strict type system, four-valued logic, and the syntactic distinction between dot and arrow navigation create a high barrier to entry. Developers face particular difficulty when constructing constraints with complex logical quantifiers or deeply nested collection operations, often leading to semantic errors [18].

*YueLou Ding is supported by China Scholarship Council and the Department of Further and Higher Education, Research, Innovation and Science.

Authors' Contact Information: Yuelou Ding, yuelou.ding.2025@mumail.ie, Department of Computer Science, Maynooth University, Maynooth, Ireland; Hao Wu, haowu@cs.nuim.ie, Department of Computer Science, Maynooth University, Maynooth, Ireland.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

Large Language Models (LLMs) offer a potential solution for automating the translation from natural language to OCL [1, 22]. Yet, deploying LLMs for formal constraint synthesis introduces the problem of hallucination[19]. These hallucinations are not merely syntactic noise; they manifest as structural and semantic defects. Structural hallucinations produce invalid OCL expressions, such as referencing undefined properties or mismatching navigation syntax. More critically, semantic drift occurs when an LLM generates a syntactically well-formed constraint that is logically divergent from the user’s requirement. Common semantic failures include omitting null-dereference guards, misinterpreting association multiplicities, or imposing overly restrictive quantifier conditions. These defects can severely compromise the correctness of downstream model transformations and verification.

Current research on LLM-based OCL generation mainly targets statistical syntax accuracy. For instance, fine-tuning LLMs on OCL corpora improves benchmark pass rates [16], but treats the process as a one-shot translation and without semantic correctness checking. This is a critical limitation, but formal methods can deal with the lack of semantic checking. To achieve this, traditional OCL validation tools—such as USE or formal encodings like OCL2FOL [4] and OCL2FOL+ [8]—can assess constraint satisfaction, but they lack the capability to repair malformed specifications. When processing the irregular outputs typical of LLMs, these tools often crash or return cryptic runtime errors. They fail to provide the actionable feedback necessary for LLM to self-correct.

We propose a neuro-symbolic synthesis pipeline that couples LLM generation with a three-layer checking mechanism. Instead of a passive generate-and-test paradigm, the system employs an iterative self-correction loop. Given a UML class diagram and a natural language requirement as input, the LLM generates an OCL Abstract Syntax Tree (AST), which then undergoes sequential validation. Layer 1 enforces structural compliance via JSON Schema validation. Layer 2 performs static type checking and null-safety enforcement within the UML context. Layer 3 translates the AST into SMT formulas and performs strict semantic equivalence checking against the reference OCL AST in a benchmark. If a layer detects a violation, it synthesizes a structured diagnostic error or a formal counterexample, appending this feedback to the prompt for the next generation cycle. This loop repeats until the output passes all checks or reaches a maximum retry threshold.

The contributions of this work are summarized as follows: (1) We introduce a neuro-symbolic OCL synthesis pipeline that embeds a self-correction loop directly into the LLM inference process. This architecture couples generation with a three-layer checking mechanism, replacing passive generate-and-test paradigms with iterative refinement guided by formal counterexamples. (2) We design a bounded SMT encoding strategy for a decidable OCL subset, where OCL’s four-valued logic is represented and collection operations are correctly implemented. This encoding is defined for formal equivalence checking between LLM generated OCL ASTs and reference ASTs in our benchmark.

2 Related Work

This work mainly draws upon research in OCL constraint synthesis, formal semantic encoding, and neuro-symbolic program repair.

2.1 OCL Constraint Generation

Bridging natural language requirements and formal OCL constraints has historically relied on heuristic translation. Early template-based approaches [3] mapped natural language vocabularies to OCL constructs, offering determinism but with limited expressive power. The advent of LLMs shifted focus to neural translation, though empirical studies indicate significant limitations in direct generation. Assessments of generative AI in modeling tasks [6] reveal that

LLMs frequently produce structural and semantic defects in MDE scenarios. This problem is particularly distinct in constraint domain-specific languages, where performance falls short without iterative correction mechanisms [10].

Subsequent research has aimed at optimizing LLM inputs to improve generation quality. Fine-tuning on OCL corpora [16] improves statistical accuracy, while path-based prompt augmentation [1] addresses context window limits when processing large UML diagrams. Reverse engineering OCL from source code using open-source LLMs [19] has also been explored. Although These methodologies have improved generation fidelity, they all treat constraint synthesis as a sequence-to-sequence translation task. Consequently, they lack the capability to mathematically verify semantic equivalence or utilize logical counterexamples for interactive refinement. Our pipeline treats generation as an iterative search process, coupling the LLM with a three-layer checking mechanism (including formal verification) that actively guides the synthesis toward a semantically equivalent OCL specification.

2.2 Formal Encoding of OCL

Rigorous verification of OCL constraints requires mapping their semantics to a formal logic framework. Early attempts like OCL2FOL [4] translated OCL into first-order logic, and subsequent extensions such as OCL2FOL+ [8] explicitly modeled the four-valued logic. This completeness unavoidably result in severe formula complexity. Explicitly encoding invalid states typically requires deep conditional branching, and the functional representation of collection operations triggers state-space explosion when mapped to automated solvers, undermining both performance and decidability.

We adopt a bounded verification strategy inspired by Alloy [12] to abstract the unbounded UML object space into a finite instance domain. Within this bounded context, we differ from traditional complete encoding methods to preserve tractability. Instead of extending the type system to accommodate invalid states, we apply a Value-Condition Separation mechanism rooted in the treatment of partial functions and domain conditions [13, 23]. This decouples expression evaluation from its safety premises, reducing OCL's exception propagation to quantifier-free linear integer arithmetic (QF_LIA) and keeping the encoding within decidable fragments. Furthermore, rather than explicitly instantiating collections as containers, we abstract collection algebra into symbolic counting functions. This approach is conceptually inspired by the decision procedure for fractional collections with cardinality bounds [17], which also reduces set operations to arithmetic constraints. However, our encoding targets linear integer arithmetic and employs bounded quantifier elimination, differing from their mixed linear rational arithmetic and star operator elimination.

2.3 Neuro-Symbolic Synthesis

The Counterexample-Guided Inductive Synthesis (CEGIS) framework [20] uses counterexamples from a verifier to iteratively refine candidate programs, but its application to domain-specific modeling languages like OCL is unexplored. Recent work in Automated Program Repair (APR) integrates CEGIS with LLMs to guide generation using symbolic feedback. Orvalho et al. [15] combined MaxSAT-based fault localization with LLMs to demonstrate that counterexample feedback outperforms pure statistical generation. Maddila et al. [14] embedded static analysis and test execution feedback into a neuro-symbolic agent, improving repair success rates for imperative code.

These approaches validate the utility of neuro-symbolic loops but rely on feedback mechanisms tailored to syntactic errors or specific test case failures. They are structurally incapable of verifying the strict semantic equivalence required for formal constraints. Our pipeline addresses this by embedding an SMT solver directly into the checking mechanism. The solver acts as a discriminating component that performs formal equivalence checking. By projecting SMT model outputs into concrete object diagram states, the pipeline provides the LLM with structured diagnostic feedback specifically targeting OCL semantic challenges, such as navigation path validation and collection multiplicity constraints.

3 Methodology

3.1 Architecture Overview

The proposed pipeline is a translation and verification architecture that employs an LLM to synthesize OCL constraints from a natural language requirement and a UML class diagram. To address the syntactic ill-formedness and semantic violations generated by LLM, the architecture uses a self-correction loop. Structural and typing errors, alongside logical counterexamples, are fed back into the prompt, constraining the LLM to make targeted corrections.

As illustrated in Figure 1, the system operates as an iterative generate-verify-repair loop. The process initiates when the LLM synthesizes a candidate OCL AST from the natural language requirement and the UML class diagram. This candidate then enters a sequential verification pipeline structured as a three-layer checking mechanism.

The verification proceeds strictly in order: the AST must first pass Layer 1 structural checks before reaching Layer 2 type checking, and finally Layer 3 formal equivalence checking. If an error is captured at any layer, the pipeline halts and synthesizes a structured diagnostic error message (e.g., a schema violation, a type mismatch, or a logical counterexample). This feedback is incorporated into the prompt, triggering a targeted self-correction iteration. This cycle continues until a semantically equivalent AST is achieved or the global retry threshold Max_{retry} is exhausted. The layers function as follows:

- Layer 1 (Syntactic & Structural Enforcement): To exclude raw syntactic hallucinations and strictly confine the generation space to a decidable OCL subset, the LLM is mandated to output a serialized Abstract Syntax Tree (AST) rather than unstructured OCL text. This layer provides the LLM with the specific AST structure as a JSON schema and crucial rules in the prompt, and then intercepts malformed JSON text, missing AST fields, and schema-violating structural variants until a well-formed AST is achieved.
- Layer 2 (Static Type Checking): This layer executes syntax-directed type derivation over the well-formed AST within a typing environment initialized from the input UML class diagram. It enforces OCL typing rules to intercept undefined property or association navigations, operand type incompatibilities (including invalid operations over collection data types and arithmetic/logic mismatches), and null-safety violations, ensuring the AST is well-typed.
- Layer 3 (Formal Verification): This layer encodes the well-typed AST into SMT formulas and performs equivalence checking between the candidate AST and a reference AST in the benchmark, via bi-directional implication. If non-equivalence is detected, the backend synthesizes a precise symbolic valuation model $\mathcal{M}$ as a counterexample to guide the LLM self-correction.

3.2 Prompt-Guided Generation and Structural Validation

To improve generation quality, we apply structural and semantic guidance in the prompt. This section details this guidance and the first layer of our three-layer checking mechanism.

For structural guidance, we manually define a strongly-typed JSON Schema representing the structure of the OCL AST. Appended to the prompt, this schema structurally confines the LLM's output space to valid AST topologies. Key structural mechanisms include: (1) Polymorphic Discriminator, utilizing a discriminator field within the union type `OCLExpression` to ensure deterministic instantiation of correct AST node subclasses; (2) Closed Enumeration, defining specific fields as enumeration types (e.g., restricting `collection_kind` to "Set" or "Bag") to structurally exclude constructs outside the supported decidable subset; and (3) Recursive Definition, enabling arbitrarily nested expressions by defining AST nodes that can reference themselves or other node types.

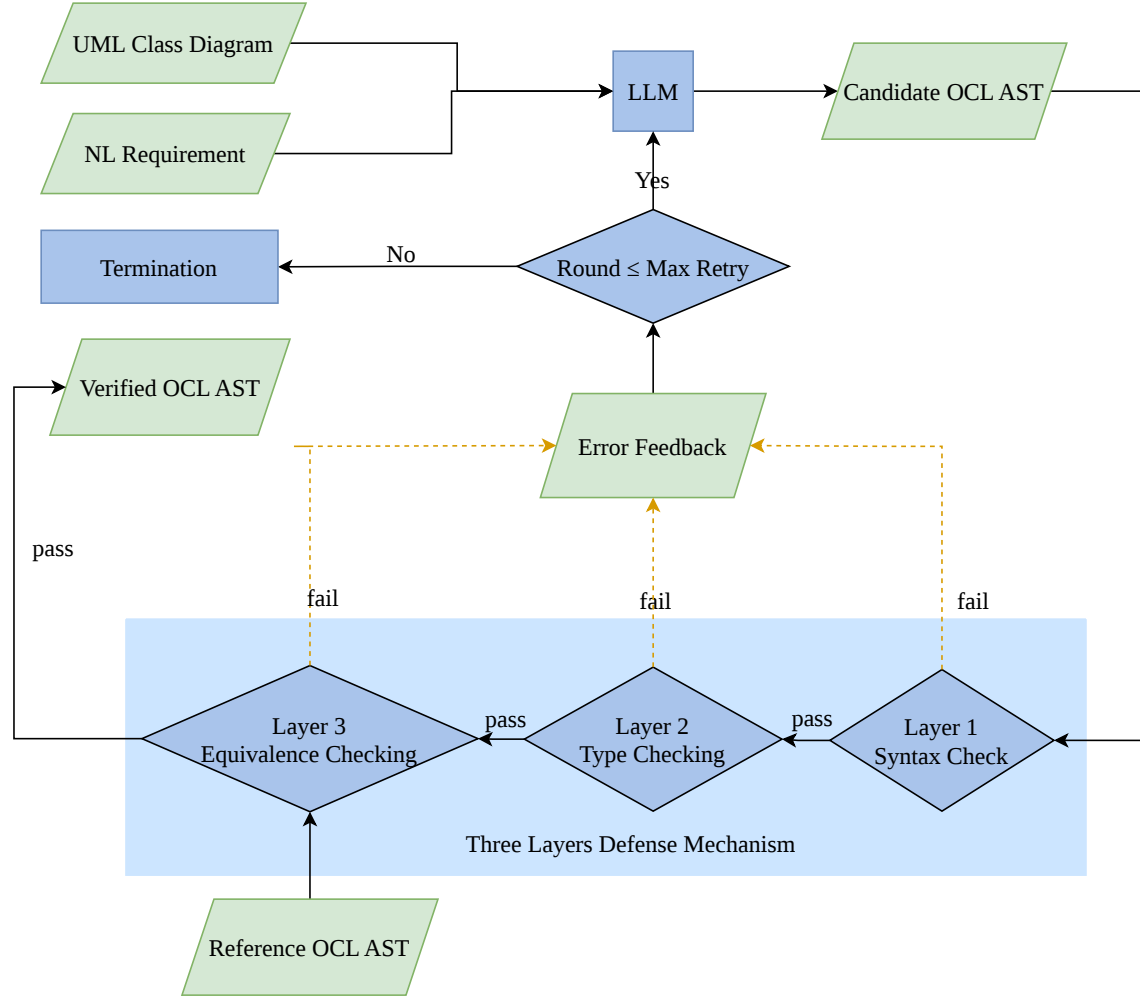


Fig. 1. Overview of the OCL synthesis pipeline with three-layer checking self-correction loop.

Since structured schemas cannot constrain semantic choices (e.g., navigation syntax), we incorporate heuristic rules via System Instruction to reduce common semantic violations. These rules mandate selecting dot (.) or arrow (→) syntax based on UML multiplicity, distinguishing mandatory associations from optional ones requiring `oclIsUndefined()` guards, and prohibiting OCL shorthand in favor of explicit iterator variable declarations to facilitate deterministic SMT encoding.

Prompt-based rules act as probabilistic guidance; LLMs may deviate from them, necessitating a posteriori checking. Layer 1 of our checking mechanism undergoes strict nominal typing and recursive schema instantiation. This establishes a deterministic structural guard, intercepting discriminator mismatches, enumeration violations, and structural integrity violations (e.g., missing required fields). Upon checking failure, the system triggers a self-correction loop, feeding the precise validation error back to the LLM.

3.3 Static Semantic Checking

While Layer 1 guarantees a structurally well-formed AST, it remains semantically agnostic regarding OCL typing rules and the UML class diagram. Layer 2 performs static type derivation and conformance checking over the AST, ensuring it is well-typed and conforms to the input UML class diagram. The checker operates via syntax-directed recursive traversal, computing the type of each expression while enforcing typing rules. It relies on two interconnected mechanisms:

- **Context-Bound Typing Environment:** To initialize the semantic context, we construct a typing environment, denoted as Γ , which maps variables to their corresponding OCL types τ . The environment is initialized by binding the implicit variable `self` to the UML context class, denoted as C_{ctx} . Property navigation is resolved via a model lookup function $\delta(C, p)$, which checks whether a class C has a property p within the UML context. If $\delta(C, p)$ fails, a navigational violation is raised as a type error.
- **Inductive Type Derivation:** The derivation algorithm computes the type of a parent node inductively based on the derived types of its children. To construct expressions with local bindings (e.g., `IteratorExpression`, `LetExpression` defined in our JSON schema), the algorithm extends the current environment Γ by binding the iterator or let variable v to its derived type τ_v . This extended environment is recursively passed to the body expression, after which the original environment is restored to ensure lexical scoping.

During this derivation process, strict OCL type conformance rules are enforced. The checker rejects undefined navigations ($\delta(C, p)$ failure), operand type incompatibilities (adhering to OCL's implicit widening coercion but rejecting invalid type combinations), iterator body mismatches (e.g., non-Boolean bodies for `select`), and declaration-assignment conflicts in `let` expressions. By intercepting these static semantic errors, Layer 2 guarantees that the AST forwarded to the SMT encoding phase is type-sound and UML-context-conformant.

3.4 Bounded SMT Encoding & Verification

Having derived a type-sound and UML-context-conformant AST through the preceding static checks, Layer 3 translates the verified OCL AST into a set of SMT formulas. The fundamental translation challenge lies in bridging the semantic gap between OCL's expressive constructs—specifically, first-order quantification over unbounded collections and the propagation of exceptional values (`null` and `invalid`)—and the solving logic of SMT solvers. To ensure the termination of the verification process, Layer 3 introduces a bounded encoding technique. This technique has three features: (1) **Finite Domain Instantiation:** mapping UML classes to finite datatypes with explicit null sentinels to establish a bounded object universe; (2) **Collection Abstraction via Counting Functions:** representing collections as mathematical functions that map instances to their occurrence counts, which allows operations like `size` and `sum` to be encoded as linear integer arithmetic constraints, thereby avoiding non-linear SMT formulas; and (3) **Propositional Expansion of Quantifiers:** eliminating inner OCL quantifiers (e.g., `forall`, `exists`) by explicitly unrolling them into finite conjunctions or disjunctions over the instance domain, converting the problem into quantifier-free SMT logic.

3.4.1 Decidable OCL Verification Subset. Standard OCL operates over potentially unbounded domains and includes constructs requiring complex ordering or underspecified selection, which conflict with efficient and decidable SMT encoding. We define a decidable OCL subset, dOCL, to ensure that all generated constraints can be faithfully mapped to SMT formulas, which is shown in Table 1. The corresponding SMT encoding strategies for these constructs are elaborated in the remainder of this section. Ordered constructs (e.g. Sequence) which require complex index axiomatization, and

underspecified selection ($\rightarrow\text{any}()$) which introduces non-determinism are not supported in this work. Also, the current subset is strictly confined to invariants; Pre- and post-conditions are excluded because their evaluation inherently relies on state-transition semantics (e.g., @pre expressions) that exceed the capacity of the stateless, single-snapshot bounded encoding defined in this work.

Table 1. Included constructs in the dOCL subset.

Category	Supported Constructs
Types & Literals	Integer, Real, Boolean, String
Logical Ops	and, or, xor, not, implies
Relational & Arithmetic	$=, <>, <, >, <=, >=, +, -, *, /, \text{abs}()$
Navigation	Dot ($.$) for single objects; Arrow ($\rightarrow$) for collections
Collections	Set, Bag literals
Iterators	forAll, exists, select, reject, collect, isUnique ¹
Collection Ops	size, isEmpty, notEmpty, includes, excludes, includesAll, excludesAll, sum, count, asSet, asBag, union, intersection
Control Flow	if-then-else, let-in
Null Safety	oclIsUndefined()

3.4.2 Bounded Structural Context Encoding. To establish a decidable foundation for OCL verification, the first core mechanism addresses the encoding of the UML class diagram. The unbounded object space of each UML class C is abstracted into a finite instance domain. Specifically, C is mapped to an SMT algebraic datatype consisting of k distinct valid constructors ($c_0, c_1, \dots, c_{k-1}$), where k denotes the upper bound of instances of C (the verification scope). To faithfully encode OCL's strict null-propagation semantics without relying on external exception handling, we extend the instance domain of each class C with a designated null sentinel. This inclusion treats null as a first-class citizen, enabling null-dereference checks to be expressed as native equality constraints.

Within this bounded context, structural features and associations are encoded as uninterpreted functions (UFs). A single-valued attribute a of class C with type T is mapped to a unary UF $f_a : C \rightarrow T$. Similarly, single-valued associations are encoded as UFs $f_{\text{assoc}} : C_{\text{src}} \rightarrow C_{\text{tgt}}$. For multi-valued associations (typed as Set or Bag), the encoding employs a cardinality-based abstraction: a multi-valued association assoc is mapped to a binary counting function $f_{\text{assoc_cnt}} : C_{\text{src}} \times C_{\text{tgt}} \rightarrow \text{Int}$. The return value represents the number of the link between source instance src and target instance tgt . This abstraction reduces collection algebraic operations to linear integer arithmetic, preserving decidability.

To ensure these UFs conform to UML structural semantics, a set of global axioms is synthesized. OCL's invalid exceptions are captured during AST translation via safety constraints. The generated axioms ensure three critical properties: (1) Default Values: UFs map null sentinels to default values (e.g., $f_a(\text{null}_C) = 0$, $f_{\text{assoc}}(\text{null}_{\text{src}}) = \text{null}_{\text{tgt}}$, $f_{\text{assoc_cnt}}(\text{null}_{\text{src}}, \text{tgt}) = 0$); (2) Multiplicity Bounds: $f_{\text{assoc_cnt}}(\text{src}, \text{tgt}) \geq 0$, constrained to 0 or 1 for unique presence in Set semantics and bounded by k for Bag semantics; (3) Mandatory Constraints: For $[1..1]$ associations, the axiom $\forall \text{src} \in C_{\text{src}} \setminus \{\text{null}_{\text{src}}\}, f_{\text{assoc}}(\text{src}) \neq \text{null}_{\text{tgt}}$ is statically expanded into quantifier-free ground clauses.

¹Multi-variable iterator forms (e.g., $\rightarrow\text{forAll}(x, y \mid \dots)$) are supported for forAll, exists, and isUnique; select, reject, and collect are restricted to single-variable iterations to preserve algebraic soundness in the current implementation.

3.4.3 Expression Translation & Safety Mechanism. Following the establishment of the structural UFs and their well-formedness axioms, the subsequent challenge lies in translating OCL expression into SMT formulas. OCL distinguishes between null and invalid. To represent OCL’s four-valued logic in two-valued SMT solver, the translation mechanism employs a value-condition separation strategy. Instead of returning a single SMT expression, the translator decouples expression evaluation from its safety premises, yielding a binary tuple (v, Φ) for each AST node. Specifically, v denotes the computed SMT value representing the node’s result under the optimistic assumption that no exceptions occur; Φ is a set of SMT Boolean formulas representing the safety conditions. For an arbitrary expression ϵ , we denote its associated safety set as Φ_ϵ . For example, translating the property access `self.manager` yields the tuple $(f_{manager}(self), \{self \neq null_C\})$. Here, v is the navigation result, and Φ contains the null-dereference guard. These safety conditions are propagated upwards through the AST, aggregating into a final safety set that ensures the validity of the entire expression.

Safety conditions are generated and propagated according to the specific semantics of each AST node type. Safety generation occurs at navigation nodes: (1) Property and Association Navigation generates a null-dereference guard $v_{source} \neq null_C$, unless the source is statically guaranteed non-null (derived from a chain of mandatory [1..1] associations rooted at `self`) or the navigated association itself has multiplicity [1..1]; (2) Division generates the condition $v_{right} \neq 0$.

Safety conditions propagate upwards along the AST, merging results from the left and right sub-trees into their parent node. This merge follows specific rules dictated by the parent node’s semantics. Let Φ_L, v_L and Φ_R, v_R denote the safety sets and computed values of the left and right operands, respectively. The propagation rules are defined as follows:

- **Unary Operations (not, unary -):** The safety set is directly propagated without modification.
- **Arithmetic and Relational Operations:** The safety sets are combined via unconditional union: $\Phi = \Phi_L \cup \Phi_R$.
- **Logical Operations with Short-Circuit Semantics:** The translator synthesizes composite safety formulas to correctly handle cases where one operand’s invalidity is masked by the other operand’s value (e.g., `false` and `invalid` yields `false`). and yields $\Phi = \{((\bigwedge_{\phi \in \Phi_L} \phi) \wedge \neg v_L) \vee ((\bigwedge_{\phi \in \Phi_R} \phi) \wedge \neg v_R) \vee ((\bigwedge_{\phi \in \Phi_L} \phi) \wedge (\bigwedge_{\phi \in \Phi_R} \phi))\}$; or yields $\Phi = \{((\bigwedge_{\phi \in \Phi_L} \phi) \wedge v_L) \vee ((\bigwedge_{\phi \in \Phi_R} \phi) \wedge v_R) \vee ((\bigwedge_{\phi \in \Phi_L} \phi) \wedge (\bigwedge_{\phi \in \Phi_R} \phi))\}$. `implies` yields $\Phi = \{(\bigwedge_{\phi \in \Phi_L} \phi) \wedge (v_L \rightarrow (\bigwedge_{\phi \in \Phi_R} \phi))\}$. These formulas faithfully implement OCL’s short-circuit semantics by enforcing an operand’s validity only when its value is essential to the result.
- **Conditional Expression:** The combined safety is $\Phi = \Phi_{cond} \cup \{\text{If}(v_{cond}, \bigwedge_{\phi \in \Phi_{then}} \phi, \bigwedge_{\phi \in \Phi_{else}} \phi)\}$.
- **Let Expression:** $\Phi = \Phi_{val} \cup \Phi_{body}$.

The operation `oclIsUndefined()` explicitly consumes safety conditions rather than propagating them, converting potential exceptions into deterministic Boolean values. The translation of $\epsilon.\text{oclIsUndefined}()$ returns the value $\neg(\bigwedge_{\phi \in \Phi_\epsilon} \phi) \vee (v_\epsilon = null_C)$ with an empty outgoing safety set. Through this mechanism, the translator faithfully encodes OCL’s strict exception propagation semantics without departing from the quantifier-free fragments of the SMT solver.

3.4.4 Collection Operations via Counting Functions. While the translation mechanism handles atomic evaluations, OCL’s collection operations introduce first-order quantification over unordered multisets. Instead of explicitly instantiating collections as container objects—a functional representation prone to severe state space explosion [4]—our pipeline represents them predicatively. Inspired by the relational tuple flattening in Alloy [12], a collection is modeled as a reference tuple $\mathcal{R} = (r, cnt, \mathcal{D}_{elem}, eval, \Phi, isSet)$, where:

- $r \in \mathcal{D}_{root}$ is the root instance anchoring the collection (e.g., the `self` variable).

- $cnt : \mathcal{D}_{root} \times \mathcal{D}_{elem} \rightarrow \mathbb{Z}$ denotes the occurrence count of element $e \in \mathcal{D}_{elem}$ in the collection.
- $\mathcal{D}_{elem}$ is the bounded instance domain of the collection's element type, inclusive of the null sentinel $null_{elem}$.
- $eval$ is an optional deferred evaluation mechanism used for implicit and explicit collect operations. It is detailed later in this section.
- Φ is the accumulated safety condition set propagated from the collection's source expressions.
- $isSet$ denotes whether the collection adheres to Set or Bag semantics. Note that Sequence semantics is outside the scope of our decidable subset.

Standard OCL collection operations are mapped to algebraic manipulations of the cnt function. Because $\mathcal{D}_{elem}$ is finite, these operations are statically expanded into quantifier-free SMT expressions. Aggregations like $size()$ translate to $\sum_{e \in \mathcal{D}_{elem}} cnt(r, e)$. Set/Bag algebra like $union$ synthesizes a new counting function: $cnt_{new}(r, e) = \max(cnt_1(r_1, e), cnt_2(r_2, e))$ for Set semantics, or $cnt_1(r_1, e) + cnt_2(r_2, e)$ for Bag semantics. Semantic coercion via $asSet()$ is achieved through algebraic truncation: $cnt_{new}(r, e) = \text{If}(cnt_{old}(r, e) > 0, 1, 0)$.

In OCL, navigation from a collection invokes an implicit collect operation. Attribute navigation is handled via deferred evaluation (binding to $eval$), while association navigation synthesizes a nested counting function immediately. For a target instance tgt , the count aggregates contributions from source instances src :

$$cnt_{new}(r, tgt) = \sum_{src \in \mathcal{D}_{src} \setminus \{null_{src}\}} \text{If}(v_{nav}(src) = tgt, cnt_{src}(r, src), 0) \quad (1)$$

where $v_{nav}(src)$ denotes the navigation result. Null suppression is implicitly enforced by excluding $null_{tgt}$ from the target domain $\mathcal{D}_{tgt}$, ensuring $v_{nav}(src) = null_{tgt}$ never satisfies the equality condition for any valid tgt .

3.4.5 Bounded Quantifier Elimination. OCL's iterator expressions inherently encode first-order logical quantifiers over collections. To guarantee decidability and tractable solving performance, we employ a bounded quantifier elimination strategy that statically expands first-order quantifiers into quantifier-free propositional logic at compile time. The iterator variable x iterates over the entire bounded instance domain $\mathcal{D}_{elem}$ (of size $k + 1$, inclusive of the null sentinel $null_{elem}$). Although null is present in the domain, structural axioms enforce $cnt(r, null_{elem}) = 0$, ensuring it is automatically excluded by the membership guard. Membership is defined as:

$$x \in S \triangleq cnt(r, x) > 0 \quad (2)$$

For a single-variable iterator with body $P(x)$, the first-order quantifiers are eliminated and expanded as finite propositional connectives. To preserve vacuous truth when the collection is empty or an element is not a member, the universal quantifier expands using implication, while the existential quantifier expands using conjunction:

$$S \rightarrow \text{forall}(x \mid P(x)) \triangleq \bigwedge_{e \in \mathcal{D}_{elem}} (e \in S \implies P(e)) \quad (3)$$

$$S \rightarrow \text{exists}(x \mid P(x)) \triangleq \bigvee_{e \in \mathcal{D}_{elem}} (e \in S \wedge P(e)) \quad (4)$$

The use of implication in forall is semantically critical: it correctly implements the OCL standard semantics where empty collections yield true (vacuous truth). Using conjunction ($\wedge$) instead would incorrectly require all domain elements to be members of S , fundamentally altering the universal quantification semantics.

On the other hand, the existential quantifier expands using conjunction ($\wedge$) within each disjunct. If implication ($\implies$) were used for exists instead, i.e., $\bigvee_{e \in \mathcal{D}_{elem}} (e \in S \implies P(e))$, then for any element $e \in \mathcal{D}_{elem}$ with $e \notin S$, the

implication $e \in S \implies P(e)$ would be vacuously true (since the antecedent $e \in S$ is false). Because $\mathcal{D}_{elem}$ invariably contains elements not in S (at minimum the null sentinel, for which $cnt(r, null_{elem}) = 0$), at least one disjunct would always evaluate to true, rendering the entire expression trivially satisfied regardless of whether any member of S actually satisfies P . This would destroy the requirement that a satisfying instance must genuinely exist within S .

Let Φ_{coll} denote the set of safety conditions propagated from the source expressions of S . For each instance $e \in \mathcal{D}_{elem}$, evaluating the body $P(e)$ may trigger an OCL invalid exception, generating a set of instance-level safety conditions Φ_e . We enforce a guarded propagation mechanism where the safety contribution of each instance is strictly conditioned on its membership in S . The final safety set of the iterator expression, denoted as Φ_{iter} , is synthesized as:

$$\Phi_{iter} = \Phi_{coll} \cup \bigcup_{e \in \mathcal{D}_{elem}} \left\{ (e \in S) \implies \bigwedge_{\phi \in \Phi_e} \phi \right\}$$

Here, ϕ represents an individual safety condition. The membership guard ($e \in S$) ensures that safety conditions for elements not in the collection (including $null_{elem}$) default to true, preventing spurious global safety failures.

Based on these mechanisms, specific iterators are encoded as follows: `forAll` and `exists` are expanded into boolean conjunctions or disjunctions. `select` and `reject` yield a new collection $\mathcal{R}_{new}$ by synthesizing a new counting function conditionally: $cnt_{new}(r, e) \triangleq \text{If } (e \in S \wedge P(e), cnt_{old}(r, e), 0)$ for `select`, and similarly with $\neg P(e)$ for `reject`. `isUnique` expands into pairwise inequality constraints over the domain, optimized by index constraints $i < j$, with body safety conditions Φ_{e_i}, Φ_{e_j} included in the antecedent to prevent undefined comparisons. `collect` employs a deferred evaluation strategy, binding the unevaluated body AST to the *eval* component until consumed by a downstream operation.

For multi-variable iterators such as `S->forAll(x, y | P(x, y))`, the translator explicitly enumerates the Cartesian product $\mathcal{D}_{elem} \times \mathcal{D}_{elem}$. The membership guard becomes a composite conjunction $e_x \in S \wedge e_y \in S$, and the safety conditions $\Phi_{(e_x, e_y)}$ for the body evaluation must be conjoined in the antecedent. This systematic expansion ensures that multi-variable quantifiers are also fully reduced to quantifier-free ground formulas.

3.4.6 Equivalence Verification & Counterexample Extraction. Following the translation into SMT formulas, the final phase of Layer 3 verifies the semantic equivalence between the LLM-generated constraint and the reference specification in our benchmark. Let $null_{ctx}$ denote the null sentinel for the context class C_{ctx} . For any constraint source $x \in \{Ref, LLM\}$, where *Ref* denotes the reference specification (the correct OCL constraint) and *LLM* denotes the LLM-generated candidate, the universally quantified assertion is formally defined as:

$$\Psi_x = \forall s, (s \in C_{ctx} \wedge s \neq null_{ctx}) \implies \left(v_x \wedge \bigwedge_{\phi \in \Phi_x} \phi \right)$$

where v_x denotes the SMT-encoded logic expression for constraint x (the AST root's semantic value), and Φ_x represents the propagated safety condition set. The quantifier variable s is a single SMT constant of sort C_{ctx} , not a recursive enumeration over class instances. The safety conditions $\phi \in \Phi_x$ are conjoined within the implication consequent, ensuring the constraint v_x is only evaluated when all dereferences are valid.

Before evaluating equivalence, the pipeline must disambiguate semantic defects in the reference specification itself, as comparing against an inherently flawed reference is meaningless. These prerequisite checks are executed incrementally within a single SMT Solver instance via push/pop operations: (1) **Safety Check:** The solver verifies the satisfiability of $(s \neq null_{ctx}) \wedge \neg \left(\bigwedge_{\phi \in \Phi_{Ref}} \phi \right)$. If sat, the reference inherently triggers potential invalid exceptions and returns

INVALID_REF. (2) Vacuity Check: The solver checks the satisfiability of Ψ_{Ref} against the UML structural axioms. If unsat, the reference is structurally unsatisfiable within the bounded scope, and verification terminates early as INVALID_REF.

If the reference specification passes both checks, the pipeline proceeds to bi-directional implication checking to determine the semantic relation between Ψ_{Ref} and Ψ_{LLM} . The equivalence verification is formalized as two entailment queries:

(1) Strengthening Detection: Check whether $\Psi_{Ref} \wedge \neg\Psi_{LLM}$ is satisfiable. This formula is logically equivalent to $\neg(\Psi_{Ref} \implies \Psi_{LLM})$, directly testing the failure of the forward implication. If sat, a state exists that satisfies the reference specification but violates the LLM constraint, indicating the LLM constraint is strictly stronger (overly restrictive). If Ψ_{LLM} is unsat while Ψ_{Ref} is satisfiable, this condition trivially holds (any model of Ψ_{Ref} violates the unsatisfiable Ψ_{LLM}), correctly identifying the LLM constraint as Strengthened. (2) Weakening Detection: Check whether $\Psi_{LLM} \wedge \neg\Psi_{Ref}$ is satisfiable. Similarly, this corresponds to $\neg(\Psi_{LLM} \implies \Psi_{Ref})$, testing the failure of the reverse implication. If sat, a state exists that satisfies the LLM constraint but violates the reference, indicating the LLM constraint is strictly weaker (overly permissive).

Based on the satisfiability results, the semantic relation is classified: EQUIVALENT (both checks unsat), STRENGTHENED (only strengthening check sat), WEAKENED (only weakening check sat), INCOMPARABLE (both checks sat), or TIMEOUT (either check returns unknown). For evaluation purposes, only EQUIVALENT is deemed successful; all non-equivalent outcomes are uniformly classified as failures.

Upon non-equivalence, the pipeline extracts a concrete counterexample from the SMT model to drive the self-correction loop. The solver’s satisfying assignment is retrieved and projected onto the bounded object diagram: all k context-class instances are fully materialized, with every attribute and association evaluated via SMT’s model completion to ensure a concrete, deterministic state snapshot. A deliberate design choice is made to feed the complete k -instance snapshot back to the LLM, rather than isolating a minimal counterexample. Although this increases prompt length, the full contextual view allows the LLM to deduce why the constraint failed and enables more targeted repairs in subsequent iterations.

3.5 Case Study

This section illustrates the formal encoding from OCL AST to Z3 constraints using a non-trivial constraint (Listing 1) that exercises core features of the translation scheme: set-valued navigation, iterator-based projection, and terminal aggregation.

Listing 1. OCL constraint for order total price calculation

```
self.totalPrice = self.items->collect(i | i.quantity * i.product.price)->sum()
```

3.5.1 UML Class Diagram and Bounded Domain. The UML class diagram underlying Listing 1 comprises three classes: Order (with a Real attribute totalPrice), LineItem (with an Integer attribute quantity), and Listing (with a Real attribute price). The associations include a set-valued association Order.items : Set(LineItem) and a mandatory single-valued association LineItem.product : Listing[1..1].

Under the bounded verification scope $k = 3$ (as defined in Section 3.4.2), each UML class C is mapped to a Z3 algebraic datatype (Datatype) consisting of k valid instance constructors (e.g., lineitem_0, lineitem_1, lineitem_2) plus one null sentinel null_C (e.g., lineitem_null). This defines the bounded instance domain $\mathcal{D}_C$ of size $k + 1$ for each class.

3.5.2 *Structural Uninterpreted Functions and Axioms.* Following Section 3.4.2, attributes and associations are encoded as Z3 uninterpreted functions (UFs). For this case study, the following UF are introduced:

- *Attribute UFs:* $f_{totalPrice} : \text{Order} \rightarrow \text{Real}$, $f_{qty} : \text{LineItem} \rightarrow \text{Int}$, $f_{price} : \text{Listing} \rightarrow \text{Real}$.
- *Single-valued navigation UF:* $f_{product_nav} : \text{LineItem} \rightarrow \text{Listing}$.
- *Counting function:* $f_{items_cnt} : \text{Order} \times \text{LineItem} \rightarrow \text{Int}$, denoting the occurrence count of links between Order and LineItem instances.

Synthesized global axioms enforce three critical properties:

- (1) *Static Default Mapping:* Ensures UF totality (e.g., $f_{qty}(\text{null}_{\text{LineItem}}) = 0$, $f_{product_nav}(\text{null}_{\text{LineItem}}) = \text{null}_{\text{Listing}}$, $f_{items_cnt}(\text{null}_{\text{Order}}, \text{tgt}) = 0$).
- (2) *Multiplicity Bounds:* Enforces Set semantics via $0 \leq f_{items_cnt}(\text{src}, \text{tgt}) \leq 1$.
- (3) *Mandatory Constraints:* For [1..1] associations, statically expanded as $\bigwedge_{j=0}^{k-1} f_{product_nav}(\text{lineitem_j}) \neq \text{null}_{\text{Listing}}$.

These axioms ensure structural well-formedness. OCL's invalid exceptions are not modeled structurally; they are handled via the value-condition separation strategy (Section 3.4.3).

3.5.3 Step-by-Step Encoding.

Step 1: Source Navigation (`self.items`). Following the tuple notation $\mathcal{R} = (r, cnt, \mathcal{D}_{elem}, eval, \Phi, isSet)$ defined in Section 3.4.4, navigating `self.items` produces the collection reference:

$$\mathcal{R}_{items} = (\text{self}, f_{items_cnt}, \mathcal{D}_{\text{LineItem}}, \text{None}, \emptyset, \text{True})$$

Here, `self` anchors the source, f_{items_cnt} is the counting function, and $isSet = \text{True}$ enforces Set semantics. The safety set $\Phi = \emptyset$ since `self` is guaranteed non-null.

Step 2: Deferred Collect (`->collect(i | i.quantity * i.product.price)`). The collect iterator defers materialization, preserving f_{items_cnt} and $\mathcal{D}_{\text{LineItem}}$ but binding the projection body into the *eval* component:

$$\mathcal{R}_{mapped} = (\text{self}, f_{items_cnt}, \mathcal{D}_{\text{LineItem}}, \Lambda, \emptyset, \text{False})$$

The deferred closure is $\Lambda = (e_{body}, i, \mathcal{E})$, where e_{body} is the AST of `i.quantity * i.product.price` and $\mathcal{E}$ is the lexical environment. The projected value for any $e \in \mathcal{D}_{\text{LineItem}}$ is determined by evaluating e_{body} under $\mathcal{E}[i \mapsto e]$ when a downstream terminal operation forces materialization. The *isSet* flag is set to *False*, reflecting OCL's semantic demotion to Bag after collect, bounding multiplicity by k rather than 1.

Step 3: Terminal Sum (`->sum()`). The `sum()` operation forces materialization of Λ . For each $e \in \mathcal{D}_{\text{LineItem}}$, evaluating e_{body} under $\mathcal{E}[i \mapsto e]$ yields a symbolic value:

$$v_e = f_{qty}(e) \times f_{price}(f_{product_nav}(e))$$

and a set of safety conditions guarding against null-dereference:

$$\Phi_e = \{e \neq \text{null}_{\text{LineItem}}, f_{product_nav}(e) \neq \text{null}_{\text{Listing}}\}$$

To handle invalid projections, the effective count is defined as:

$$cnt_{eff}(e) = f_{items_cnt}(\text{self}, e) \cdot \text{If}\left(\bigwedge_{\phi \in \Phi_e} \phi, 1, 0\right)$$

using Z3's if-then-else (ITE) construct. This yields $cnt_{eff}(e) = 0$ when the safety conditions in Φ_e are not fully satisfied, mirroring OCL's null-safe skipping.

Direct multiplication between the integer term $cnt_{eff}(e)$ and the arithmetic term v_e yields non-linear arithmetic; since $0 \leq cnt_{eff}(e) \leq k$, it is linearized via stepwise conditional additions:

$$\sum_{j=1}^k \text{If}(cnt_{eff}(e) \geq j, v_e, 0)$$

This demotes the aggregation into the decidable quantifier-free linear arithmetic fragment (QF_LIA/LRA).

Unified Formula. Combining safety-gated evaluation and bounded linearization, the terminal $\rightarrow\text{sum}()$ translates into the following quantifier-free Z3 assertion:

$$f_{totalPrice}(self) = \sum_{e \in D_{LineItem}} \sum_{j=1}^k \text{If}\left(f_{items_cnt}(self, e) \geq j \wedge \left(\bigwedge_{\phi \in \Phi_e} \phi\right), v_e, 0\right) \quad (5)$$

This algebraically closes the encoding pipeline: the original OCL constraint is faithfully reduced to a single assertion satisfying both OCL's type/null semantics and SMT decidability boundaries.

4 Evaluation

The evaluation is structured around three primary research questions (RQs):

- RQ1 (Effectiveness): What is the end-to-end equivalence success rate of the pipeline across varying constraint complexities, and what are the primary failure modes?
- RQ2 (Component Necessity): How do the three-layer checking mechanism and the counterexample-guided self-correction loop contribute to the final equivalence rate?
- RQ3 (Efficiency & Sensitivity): Is the bounded SMT encoding strategy computationally tractable for complex OCL constraints, and how sensitive are the verification accuracy and computational cost to the bounding scope k ?

4.1 Experimental Setup

4.1.1 Benchmark Dataset. The evaluation utilizes a benchmark suite comprising 137 reference OCL specifications across 32 UML class diagrams. These diagrams are collected from two primary sources: (1) publicly available UML/OCL models gathered from online open-source repositories and academic educational resources, adapted to align with the decidable OCL verification subset and to increase structural diversity; (2) complex, domain-specific business scenarios (e.g., IoT security, healthcare management, e-commerce processing) manually designed to evaluate constraint synthesis under realistic operational logic and cover diverse OCL constructs within the supported subset. To provide a granular assessment of formal verification capabilities, constraints are classified into two structural complexity tiers based on the syntactic features of their reference OCL ASTs:

- Single Navigation: Constraints involving dot-notation property traversal but excluding collection operations (e.g., `self.salary > 0`). This tier comprises 65 constraints, representing 47.5% of the dataset.
- Collection Algebra: Constraints involving arrow-notation iterators (e.g., `forAll`, `collect`) or aggregate operations (e.g., `sum`, `size`). This tier comprises 72 constraints, representing 52.5% of the dataset and encompasses the primary challenges for SMT encoding.

4.1.2 Implementation & Environment. The LLM backend is configured as Gemini 3.1 Pro, selected for its strong logic reasoning capabilities and extended context window. The sampling temperature is set to 0 to ensure deterministic generation, a standard practice in code synthesis [7]. The maximum self-correction retry threshold is set to 5. The Z3 SMT solver [9] is configured with a strict timeout of 60,000 ms to balance completeness with computational tractability. The bounded verification scope k is set to 3 for RQ1 and RQ2, following standard practices in Alloy-style bounded analysis [12], while RQ3 treats k as an independent variable to test its sensitivity. All metrics are collected via the pipeline’s internal instrumentation, separating encoding time from solver time. To ensure reliability of the benchmark, we perform a consistency check by encoding all reference constraints within the same UML context into a single SMT instance. The SMT solver confirms that all 32 cases are jointly satisfiable under the bounded scope $k=3$. This formally guarantees that the benchmark is free from logical contradictions, providing a mathematically valid baseline for the subsequent equivalence checking. The complete implementation of the proposed pipeline and detailed benchmark information are publicly available to facilitate reproducibility at: <https://github.com/yueloud/OVAS-OCL-Verified-Automated-Synthesis>

4.2 RQ1: End-to-End Effectiveness

Table 2 summarizes the overall semantic equivalence verification results. The pipeline achieves a strict equivalence rate of 98.54%. Non-equivalent outputs are classified into STRENGTHENED (overly restrictive) and INCOMPARABLE (cross-implication failure).

Table 2. Overall Semantic Equivalence Results by Constraint Complexity

Complexity	Eq. (%)	Str. (%)	Weak. (%)	Incomp. (%)	Timeout (%)
Single Nav.	98.46	0.00	0.00	1.54	0.00
Coll. Alg.	98.61	1.39	0.00	0.00	0.00
Overall	98.54	0.73	0.00	0.73	0.00

Failure Mode Analysis. A detailed inspection of the two non-equivalent cases reveals distinct semantic failure modes. For the STRENGTHENED case (Flight_CapacityUtilization), the LLM omits the implication antecedent entirely. The reference specification defines a conditional invariant (`self.plane.capacity > 0 implies self.passengers->size() / self.plane.capacity <= 0.9`), but the LLM generates an unconditional division constraint. Without the divisor guard, the constraint fails to evaluate to true via vacuous truth when `capacity <= 0`, making it strictly stronger than the reference.

For the INCOMPARABLE case (TransferRequest_SufficientFreeStock), the failure stems from a string literal mismatch in the antecedent. The reference requires `self.status = 'Pending'`, but the LLM synthesizes `self.status = 'valid'`. This semantic misinterpretation causes the generated constraint to apply to an incorrect set of states. It simultaneously becomes overly restrictive for states where `status = 'valid'` (enforcing stock checks that the reference does not mandate) and overly permissive for states where `status = 'Pending'` (omitting the required stock checks), resulting in a cross-implication failure.

The distribution of failure modes indicates that both Single Navigation and Collection Algebra constraints achieve high equivalence rates (98.46% and 98.61%, respectively). The failures are not caused by structural syntactic errors, as the

JSON Schema effectively directs the LLM to generate well-formed AST structures. Instead, both failures originate from semantic deduction errors during conditional reasoning, specifically the omission or misinterpretation of antecedent guards. This suggests that while structural constraints are well-managed, logical precondition synthesis remains the primary boundary for LLM-based formal constraint generation.

4.3 RQ2: Component Contribution

To quantify the necessity of each architectural component, we conduct an ablation study using the pipeline’s internal configuration switches. Table 3 contrasts the full pipeline against degraded variants using the strict equivalence rate as the core metric.

Table 3. Ablation Study on Architectural Components

Configuration	Eq. Rate (%)	Equivalent Count
Full Pipeline (Baseline)	98.54	135/137
- Pre-Verification (L1 & L2)	83.21	114/137
- CGSC Loop	94.89	130/137

4.3.1 Impact of Pre-Verification. Disabling the pre-verification components causes the equivalence rate to drop significantly to 83.21%. To ensure the experiment remains executable, this configuration retains the baseline JSON Schema structure constraint and the decidable OCL subset definition in the system prompt, preventing the LLM from generating entirely unparseable outputs. However, without static type derivation and navigation syntax checking, malformed ASTs and typing violations propagate directly to the SMT translator. This results in a surge of translation crashes and severe semantic drifts, empirically demonstrating that the Pre-Verification checkers are an indispensable filtering mechanism for rejecting ill-typed ASTs before they reach the formal backend.

4.3.2 Impact of Counterexample-Guided Self-Correction. Disabling the CGSC loop yields an equivalence rate of 94.89%, a macroscopic decrease of 3.65% compared to the full pipeline. Without counterexample-driven retries, 7 constraints fail the equivalence check. When enabled, the CGSC mechanism successfully repairs 5 of these cases, achieving a 71.4% local repair rate. The remaining 2 unrepaired constraints exceed the model’s logical reasoning boundary. As analyzed in RQ1, these persistent failures stem from specific semantic deduction errors, such as omitting division guards or mismatching string literals in conditional antecedents. Textual counterexamples alone are insufficient to bridge these semantic gaps. The primary value of Layer 3 lies in providing formal verification guarantees; the CGSC loop serves as a complementary refinement step. While its macroscopic gain is structurally limited by the baseline model’s zero-shot accuracy, its targeted repair capability on logically flawed constraints remains empirically validated.

4.4 RQ3: Efficiency and Scope Sensitivity

The computational tractability of the bounded SMT encoding and its sensitivity to the scope parameter k are evaluated. We analyze the execution time distribution and the impact of varying $k \in \{3, 4, 5\}$ on equivalence rates and solver timeouts.

4.4.1 Encoding vs. Solving Overhead. The encoding phase (AST-to-Z3 translation) exhibits a tight upper bound due to the compile-time expansion strategy. For the majority of constraints, solving time remains highly tractable. However, variance emerges primarily correlated with two factors: the nesting depth of iterators and the presence of genuine non-linear arithmetic. Constraints involving deep `collect->sum` chains are successfully demoted to quantifier-free linear arithmetic (QF_LIA/LRA) via the `_linear_multiply` expansion, maintaining the problem within decidable SMT fragments at $k = 3$.

4.4.2 Sensitivity to Bounding Scope k . Table 4 details the impact of k on verification outcomes. As k increases, the state space expands exponentially. At $k = 4$ and $k = 5$, the strict equivalence rate marginally decreases from 98.54% to 97.81%. This decrease is strictly attributed to a single solver timeout rather than the discovery of new logical counterexamples.

The timeout occurs exclusively on the `Polygon_NoThreeConsecutiveCollinear` constraint. This specific constraint involves a triple-variable iterator (`forall(p1, p2, p3 | ...)`) coupled with genuine non-linear arithmetic (cross-multiplication of coordinate differences). The translation of the triple-variable iterator explicitly enumerates the Cartesian product $D_{elem} \times D_{elem} \times D_{elem}$, causing a combinatorial explosion in the number of ground clauses. When combined with non-linear terms that resist linearization, the SMT solver exceeds the 60,000 ms threshold, rendering the equivalence query undecidable within the time limit. This isolates the computational boundary of the current bounded encoding strategy to extreme combinations of multi-variable quantification and non-linear arithmetic, while confirming its tractability for standard OCL constructs.

Table 4. Impact of Bounding Scope k on Verification Metrics

Scope k	Eq. (%)	Avg. CGSC Rounds	Avg. Solving (s)	Timeout (%)
3	98.54	0.12	0.0225	0.00
4	97.81	0.12	0.4457	0.73
5	97.81	0.12	0.4638	0.73

The bounded SMT encoding ensures high computational tractability, with an average solving time of 0.0225 seconds at $k = 3$. Increasing the scope k exponentially expands the state space, which marginally decreases the strict equivalence rate due to solver timeouts on extreme constraints. The timeout is exclusively triggered by the combinatorial explosion of triple-variable iterators coupled with non-linear arithmetic, clearly delineating the tractable boundary of the decidable OCL subset and confirming the robustness of the encoding for standard constructs.

5 Discussion

The evaluation results demonstrate the feasibility of coupling probabilistic LLM generation with deterministic formal verification through the three-layer checking mechanism. By intercepting ill-typed ASTs via static checking and guiding semantic refinement via formal counterexamples, the pipeline effectively bridges the gap between statistical synthesis and mathematical guarantees. However, the empirical findings also expose specific boundaries of this neuro-symbolic approach, particularly concerning the tractability of bounded SMT encoding and the inherent reasoning limitations of current LLM backends. These trade-offs and limitations are discussed in the subsequent sections.

5.1 Trade-offs in Bounded SMT Encoding

The proposed dOCL subset and bounded encoding strategy represent a deliberate engineering trade-off between decidability and expressiveness. By strictly excluding Sequence, OrderedSet, and non-deterministic selection operations, complex collection algebra is reduced to QF_LIA/LRA constraints based on counting functions, fundamentally guaranteeing the termination of SMT solving. When addressing the exception propagation semantics of OCL’s four-valued logic, the Value-Condition Separation mechanism decouples expression evaluation from safety condition sets. This design avoids the need to introduce complex type system extensions to handle null and invalid states, allowing standard SMT solvers to directly and faithfully encode null-dereference semantics.

5.2 Threats to Validity

This architecture mainly faces two threats to validity. (1) The bounded verification provides strict semantic equivalence guarantees only within the finite instance domain of scope k . For constraint violations requiring interactions among more than k objects, the system poses a false-negative risk. However, based on the Small Scope Hypothesis (SSH) [2], the vast majority of design defects manifest within minimal instance domains, rendering the verification results highly credible in most scenarios. (2) The current evaluation primarily validates the technical feasibility of the neuro-symbolic pipeline by comparing LLM-generated constraints against pre-existing reference specifications. But in practical engineering scenarios, such reference specifications are typically unavailable, where this architecture can not be directly utilized.

5.3 Future Work

Future work will advance along several distinct directions to address the identified limitations. For reference-free practical scenarios, the pipeline will be adjusted to repurpose Layer 3 for consistency checking. Instead of verifying equivalence against a reference, the SMT solver will assert satisfiability against UML structural axioms to detect logical conflicts or vacuous truths among multiple synthesized constraints within the same context. This adaptation requires minimal engineering effort, as the core technical asset of the pipeline—the bounded encoding mechanism translating OCL ASTs to SMT formulas—remains entirely unmodified. In addition, the expansion of the dOCL subset will explore encoding strategies for ordered collections and relative operations, and then gradually extend the scope of automated synthesis and verification from invariants to pre- and post-conditions.

Acknowledgments

Generative AI tools were only adopted for English grammar correction and paraphrasing of the manuscript text. All technical contents are independently completed and fully reviewed by the authors, who take full responsibility for the published work.

References

- [1] Seif Abukhalaf, Mohammad Hamdaqa, and Foutse Khomh. 2024. PathOCL: path-based prompt augmentation for OCL generation with GPT-4. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*. 108–118.
- [2] Alexandr Andoni, Dumitru Daniliuc, Sarfraz Khurshid, and Darko Marinov. 2003. Evaluating the “small scope hypothesis”. In *In Popl*, Vol. 2.
- [3] Imran Sarwar Bajwa, Mark Lee, and Behzad Bordbar. 2012. Translating natural language constraints to OCL. *Journal of King Saud University-Computer and Information Sciences* 24, 2 (2012), 117–128.
- [4] Bernhard Beckert, Uwe Keller, and Peter H Schmitt. 2002. Translating the Object Constraint Language into first-order predicate logic. In *Proceedings of the Second Verification Workshop: VERIFY*, Vol. 2. 02–07.
- [5] Marco Brambilla, Jordi Cabot, and Manuel Wimmer. 2017. *Model-driven software engineering in practice*. Morgan & Claypool Publishers.

- [6] Javier Cámara, Javier Troya, Lola Burgueño, and Antonio Vallecillo. 2023. On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML: JJ. Cámara et al. *Software and Systems Modeling* 22, 3 (2023), 781–793.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [8] Carolina Dania and Manuel Clavel. 2013. OCL2FOL+: Coping with Undefinedness. *OCL@ MoDELS* 1092 (2013), 53–62.
- [9] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.
- [10] David Delgado, Lola Burgueño, and Robert Clarisó. 2026. A framework for assessing the capabilities of code generation of constraint domain-specific languages with large language models. *Journal of Systems and Software* (2026), 112871.
- [11] A Definitive Guide. 2012. Object Constraint Language (OCL). *Formal Methods for Model-Driven Engineering: 12th International School on Formal Methods for the Design of Computer, Communication and Software Systems, SFM 2012, Bertinoro, Italy, June 18-23, 2012. Advanced Lectures* 7320 (2012), 58.
- [12] Daniel Jackson. 2019. Alloy: a language and tool for exploring software designs. *Commun. ACM* 62, 9 (2019), 66–76.
- [13] K Rustan M Leino and Wolfram Schulte. 2007. A verifying compiler for a multi-threaded object-oriented. *Softw. Syst. Rel. Security* 9 (2007), 351.
- [14] Chandra Maddila, Adam Tait, Claire Chang, Daniel Cheng, Nauman Ahmad, Vijayaraghavan Murali, Marshall Roch, Arnaud Avondet, Aaron Meltzer, Victor Montalvao, et al. 2026. Agentic Program Repair from Test Failures at Scale: A Neuro-symbolic approach with static analysis and test execution feedback. *IEEE Transactions on Software Engineering* (2026).
- [15] Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.
- [16] Fengjunjie Pan, Vahid Zolfaghari, Long Wen, Nenad Petrovic, Jianjie Lin, and Alois Knoll. 2024. Generative ai for ocl constraint generation: Dataset collection and llm fine-tuning. In *2024 IEEE International Symposium on Systems Engineering (ISSE)*. IEEE, 1–8.
- [17] Ruzica Piskac and Viktor Kuncak. 2008. Fractional collections with cardinality bounds, and mixed linear arithmetic with stars. In *International Workshop on Computer Science Logic*. Springer, 124–138.
- [18] Rozilawati Razali, Colin Snook, Michael Poppleton, and Paul Garratt. 2008. Usability assessment of a UML-based formal modeling method using a cognitive dimensions framework. *Human Technology: An Interdisciplinary Journal on Humans in ICT Environments* 1 (2008).
- [19] H Siala and Kevin Lano. 2025. Using LLMs to extract OCL specifications from Java and Python programs: an empirical study. *OCL '25, STAF* (2025).
- [20] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial sketching for finite programs. In *Proceedings of the 12th international conference on Architectural support for programming languages and operating systems*. 404–415.
- [21] Jos Warmer and Anneke Kleppe. 1998. *The object constraint language: precise modeling with UML*. Addison-Wesley Longman Publishing Co., Inc.
- [22] Ke Weng, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu, and Tiancheng Zhang. 2025. Autoformalization in the era of large language models: A survey. *arXiv preprint arXiv:2505.23486* (2025).
- [23] Freek Wiedijk and Jan Zwanenburg. 2003. First order logic with domain conditions. In *International Conference on Theorem Proving in Higher Order Logics*. Springer, 221–237.